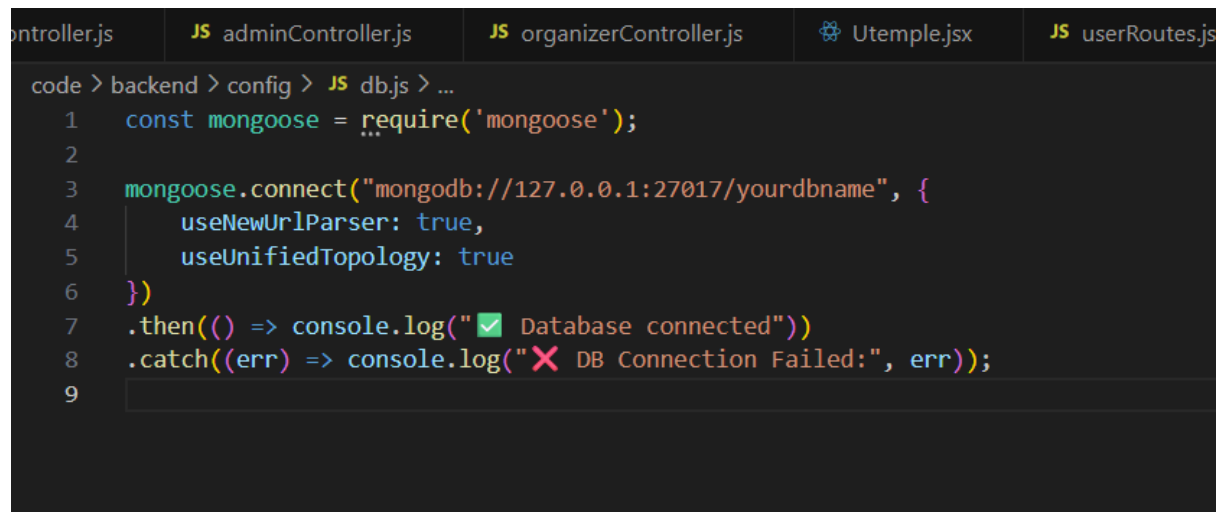


Database Connection

A screenshot of a code editor with a dark theme. The editor shows a file named 'db.js' in the 'config' folder. The code is written in JavaScript and uses the Mongoose library to connect to a MongoDB database. The code includes comments and uses emojis to indicate success or failure of the connection. The file explorer on the left shows other files like 'controller.js', 'adminController.js', 'organizerController.js', 'Utemplate.jsx', and 'userRoutes.js'.

```
code > backend > config > JS db.js > ...
1  const mongoose = require('mongoose');
2
3  mongoose.connect("mongodb://127.0.0.1:27017/yourdbname", {
4    useNewUrlParser: true,
5    useUnifiedTopology: true
6  })
7  .then(() => console.log("✅ Database connected"))
8  .catch((err) => console.log("❌ DB Connection Failed:", err));
9
```

The **DarshanEase – Temple Darshan Ticket Booking System** uses **MongoDB** as its primary database. Before performing any backend operations such as user registration, temple management, darshan slot booking, or payment processing, the application must establish a successful connection with the MongoDB database.

A dedicated database configuration file (**db.js**) is created inside the **config** folder to handle the database connection using **Mongoose**. The connection string (MongoDB URI) is stored securely in the **.env** file, ensuring sensitive information is not hardcoded into the application.

When the backend server starts, it first attempts to connect to the MongoDB database. If the connection is established successfully, the application displays a confirmation message indicating that the database is connected. If the connection fails, an appropriate error message is displayed, allowing developers to identify and resolve the issue before processing any client requests.

Establishing the database connection at the beginning of the application lifecycle ensures that all backend modules—including user authentication, temple management, darshan slot management, booking services, and payment processing—can securely access and manage the required data.

The following figure illustrates a sample database connection implementation using **Mongoose** in the **DarshanEase** backend.